Machine Learning for Physics. Part II

Henri Leroy, PhD Student (ML for granular) – *henri.leroy@inria.fr*

Univ. Grenoble Alpes, INRIA, MIAI | Artificial Intelligence & Mechanics (AIM)

Program:

I. Learning operators (1-2-3)
II. Geometric Deep Learning: Symmetry (4)

Structure:

- Theory (Slides + papers)
- Practicals
- Projects (SINDy, HNN, s-PCA, PINO)







I. Operator learning.

How to learn mappings between functions?

(i.e. infinite dimensional spaces)

A. Problem definition

D: Bounded, open set.

$\mathcal{A} = \mathcal{A}(D; \mathbb{R}^{d_a}), \mathcal{U} = \mathcal{U}(D; \mathbb{R}^{d_u})$: separable Banach spaces of function taking values in $\mathbb{R}^{d_a}, \mathbb{R}^{d_u}$ resp.

$\{a_j, u_j\}_{j=1}^N$: Finite collection of observed input-output pairs*.

$\mathcal{G}^\dagger : \mathcal{A} \to \mathcal{U}$: The operator mapping the coefficients to the solutions $\quad u_j = \mathcal{G}^\dagger(a_j)$

Goal. To build an approximation of $\mathcal{G}^\dagger$ by constructing a parametric map

$$\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}, \theta \in \Theta$$

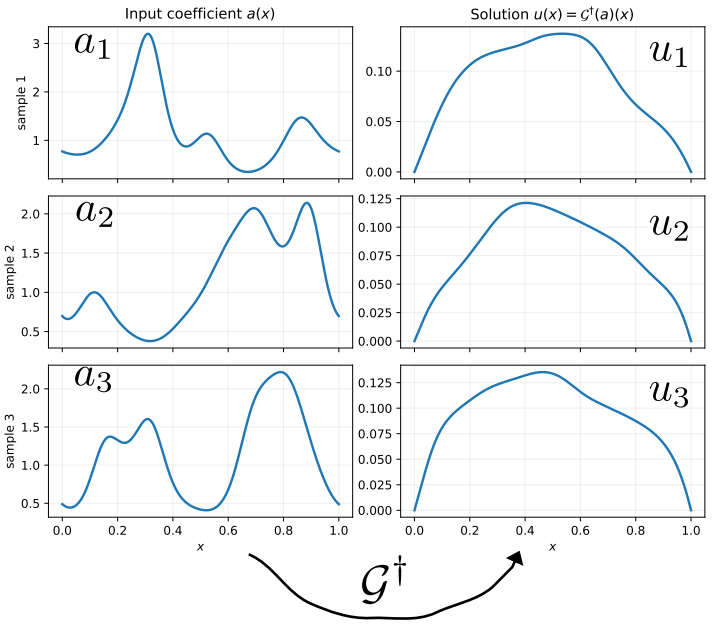
Θ : Finite-dimensional parameter space $\quad C : \mathcal{U} \times \mathcal{U} \to \mathbb{R}$: a cost functional

Minimisation problem. We seek a minimizer of the problem:

$$\min_{\theta \in \Theta} \mathbb{E}_{a \sim \mu} C(G(a, \theta), G^\dagger(a))$$

* $a_j \sim \mu$ is an i.i.d. sequence from the probability measure supported on $\mathcal{A}$

Example: *1D Darcy flow input/output*



I. Operator learning. *Why could it be interesting?*

B. Motivations

Most existing methods (FEM, FDM, PINNs...) aims to find the solution $u \in \mathcal{U}$ of a PDE for one instance of the parameter $\nu \in \mathcal{A}$

In some cases, learning an operator can drastically decrease the computational cost:

- Cheap surrogates

$$N_{\text{query}} \cdot C_{\text{solve}}$$

"Classical" numerical solver cost

- Unknown PDEs

$$N_{\text{query}} \cdot C_{\text{infer}} + C_{\text{train}} + (N_{\text{train}} \cdot C_{\text{solve}})$$

Training + Inference (+ generation) cost

Inverse problems

- Hyper-resolution

Many-query problems (UQ, optim, forecasting...)

I. Operator learning.

How to approximate an infinite dimensional map?

C. Problem formulation

Target. $\mathcal{G}^\dagger : \mathcal{A} \to \mathcal{U}$

Training data. $\{a_j, u_j\}_{j=1}^N \qquad u_j = \mathcal{G}^\dagger(a_j)$

Operator family. $\{\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}, \theta \in \Theta\}$

Goal. *Find* $\theta^* \in \text{argmin}_{\theta\in\Theta} \frac{1}{N} \sum_{j=1}^{N} ||(u_j - \mathcal{G}_\theta(a_j)||_{L^2}^2$

Q. How to learn the infinite-dimension operator?

PCA-Net

- Exploit a low-rank solution manifold.
- A network predicts a small set of coefficients

Neural Operator

- Keep spatial information throughout the network
- Parametrize non-local interactions efficiently.

I.1. PCA-Net.

How to use low-rank projection to learn an operator?

Reminder

Principal Component Analysis (PCA).

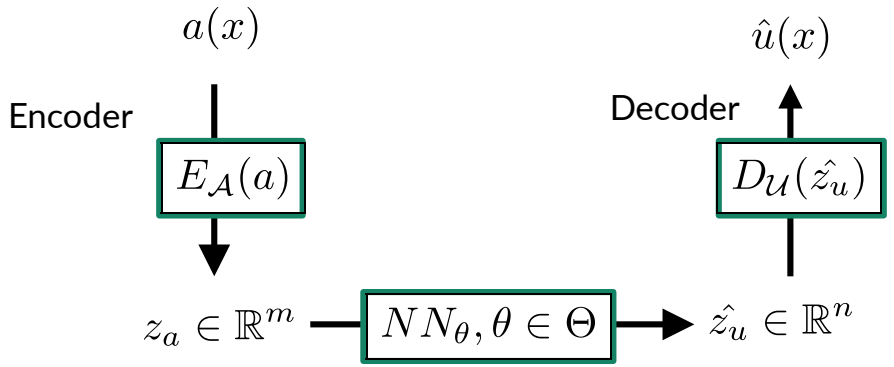
Intuition.
Find an orthonormal basis that maximizes the variance captured by projection/minimize reconstruction error.

Optimization formulation:

$$\min_{\substack{V_r \subset \mathcal{A} \\ \dim(V_r)=r}} \mathbb{E}_{a\sim\mu}\left[\|(a-\bar{a}) - P_{V_r}(a-\bar{a})\|^2\right]$$

Orthogonal projection onto the r-dimensional space

Learned operator: $\mathcal{G}_\theta = D_\mathcal{U} \circ NN_\theta \circ E_\mathcal{A}$



To be learned:

- *PCA modes: Eigenvectors of the covariance matrix (SVD)*
- *NN: Training after having computed/fixed the PCA ranks*

Q1. How to choose the number of retained modes?

Q2. How to quantify the error?

I.1. PCA-Net.

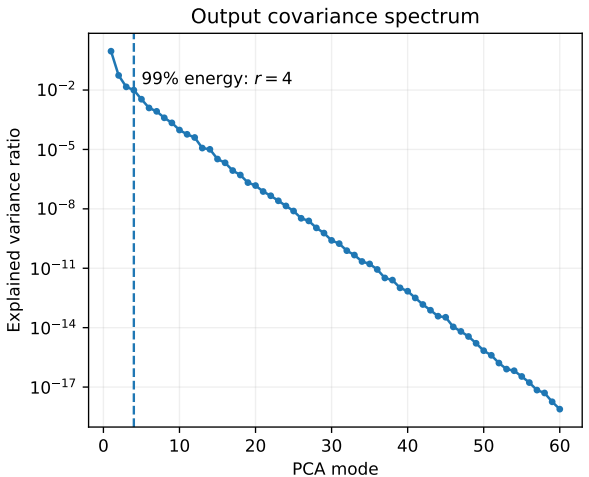
How to use low-rank projection to learn an operator?

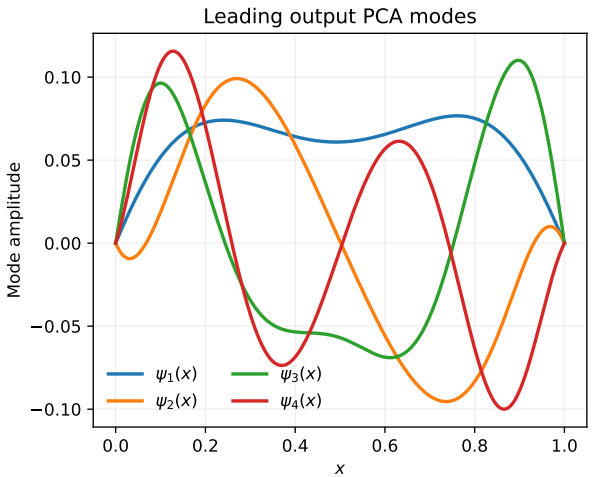
Q1. How to choose the number of retained modes?

→ **Compute the energy spectrum.**

→ **Define a threshold for $E_{\mathcal{A}}$ and $D_{\mathcal{U}}$**
e.g. 99% energy:

$$E(r) = \frac{\sum_{i=1}^{r} \lambda_i}{\sum_{i=1}^{d} \lambda_i} = \frac{\sum_{i=1}^{r} \sigma_i^2}{\sum_{i=1}^{d} \sigma_i^2}$$





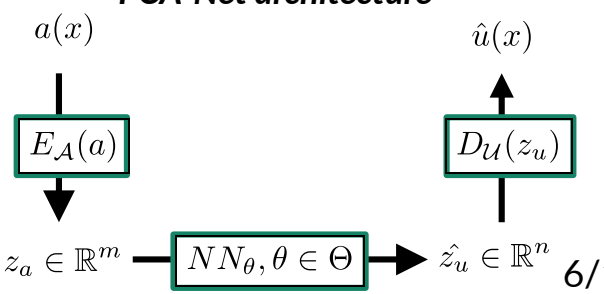
Q2. How to quantify the error?

Different sources of error:

- **Representation error:**
  Truncated modes in Encoder/Decoder
  Linear projection.
  NN capabilities.

- **Learned reduced-map error:**
  NN training

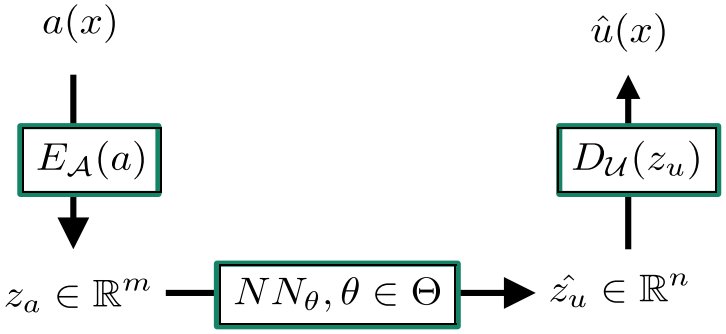
PCA-Net architecture



I.1. PCA-Net.

How to use low-rank projection to learn an operator?

PCA-Net architecture



How to.

1. Apply PCA on $\{a_j\}_{j=1}^N$ $\{u_j\}_{j=1}^N$
2. Fix the number of retained ranks for $E_{\mathcal{A}}$ & $D_{\mathcal{U}}$.
3. Train a NN on the latent mapping.

Practical tips.

- Are the input/output compressible?
- Inspect the PCA spectrum first:
  - → fast eigenvalue decay means ...
  - → slow decay indicates that ...
- Separate representation error from learning error.
- Remember the inductive bias: PCA assumes that the relevant family of fields is close to a low-dimensional linear subspace.

I. Operator learning – Discretization.

Why not simply use a standard Neural Network?

A dense Multi-Layer Perceptron (MLP) learns the mapping:

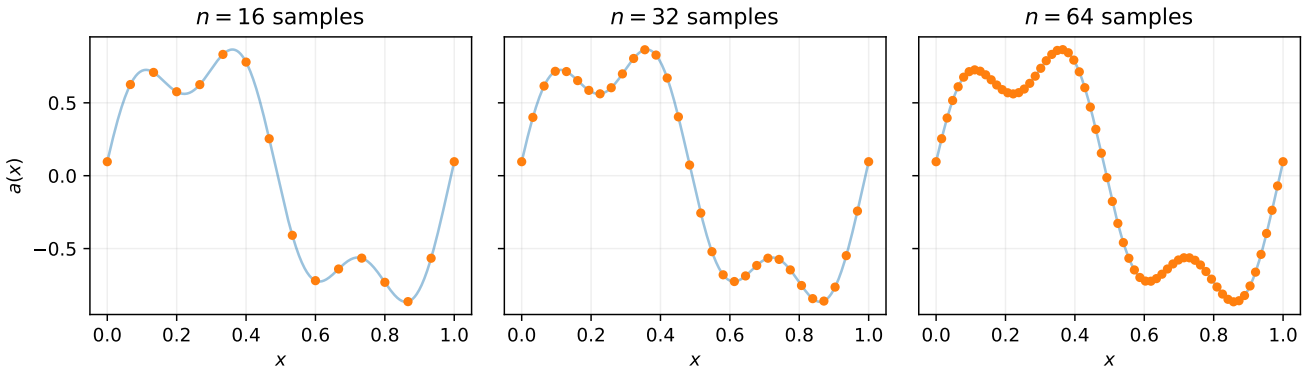
$$F_\theta : \mathbb{R}^n \to \mathbb{R}^n, \theta \in \Theta$$

Finite dimension

First, we train the MLP with 16 samples per input.

Then, we obtain 32 samples per input.

How can the MLP handle the new resolution?



On the other side, some architectures are built to be compatible with different discretizations:

$$\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}, \theta \in \Theta$$

e.g. Fourier Neural Operator (FNO)

I.2. Neural Operator. *How to approximate an infinite dimensional map?*

A. Architecture explained.

1. Lifting to a higher dimensional representation by a local pointwise transformation $P : \mathbb{R}^{d_a} \to \mathbb{R}^{d_v}$

i.e. No spatial mixing

$$v_0(x) = P(a(x))$$

2. Iterations of updates

$$v_0 \mapsto v_1 \mapsto \cdots \mapsto v_T$$

3. Projection by the local transformation $Q : \mathbb{R}^{d_v} \to \mathbb{R}^{d_u}$

$$u(x) = Q(v_T(x))$$

Note. *The neural operator can be defined independently of a particular discretization.*

Update rule:

$$v_{t+1}(x) := \sigma(W v_t(x) + (\mathcal{K}(a;\phi)v_t)(x)), \forall x \in D$$

$$(\mathcal{K}(a;\phi)v_t)(x) := \int_D \kappa(x, y, a(x), a(y); \phi) v_t(y) dy, \forall x \in D$$

Kernel integral operator

Neural network

Learnable parameters.

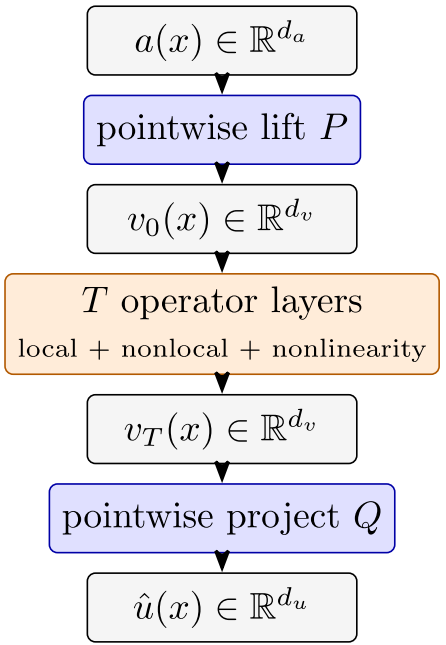
P, Q : Lifting/Projection mappings

$\{W_i, \kappa_i\}_{i=1}^T$: Weights/kernel matrices

I.2. Neural Operator.

How to compute efficiently the integral operator?

Inference process for a trained NO:



After discretization in 1D, $V_T \in \mathbb{R}^{n \times d_v}$

n : *Number of spatial points*

Update rule:

$$v_{t+1}(x) := \sigma(W v_t(x) + (\mathcal{K}(a;\phi)v_t)(x)), \forall x \in D$$

$$(\mathcal{K}(a;\phi)v_t)(x) := \int_D \kappa(x, y, a(x), a(y); \phi) v_t(y) dy, \in D$$

Kernel integral operator — *Neural network*

Q. How to compute efficiently the kernel integral operator?
How to avoid computing each (x, y) pair?

A. Depends on the structure we assume for the kernel.

I.2. Neural Operator.

How to compute efficiently the integral operator?

$$v_{t+1}(x) := \sigma(W v_t(x) + (\mathcal{K}(a;\phi)v_t)(x)), \forall x \in D$$

$$(\mathcal{K}(a;\phi)v_t)(x) := \int_D \kappa(x, y, a(x), a(y); \phi) v_t(y) dy, \in D$$

Kernel structure $\kappa(x, y)$?

- Local interactions → **Local Graph NO**

- Small number of separable components → **Low-rank NO**

- Coarse distant/accurate nearby interactions → **Hierarchical NO**

- Fourier multiplier → **Fourier NO (FNO)**

Let's parametrize the kernel in Fourier space $\mathcal{F}(\kappa_\phi) := R_\phi$

Reminder

Let's recall the **Fourier transform** $\mathcal{F}$ of a function $f : D \to \mathbb{R}^{d_v}$ and its inverse $\mathcal{F}^{-1}$

$$(\mathcal{F}f)_j(k) = \int_D f_j(x) e^{-2i\pi\langle x,k\rangle} dx$$

$$(\mathcal{F}^{-1}f)_j(x) = \int_D f_j(k) e^{2i\pi,k\rangle} dk$$

for $j = 1, \ldots, d_v$

By letting $\kappa_\phi(x, y, a(x), a(y)) = \kappa_\phi(x - y)$ and applying the convolution theorem, we find that

$$(\mathcal{K}(a;\phi)v_t)(x) = \mathcal{F}^-1(\mathcal{F}(\kappa_\phi) \cdot (\mathcal{F}v_t))(x), \forall x \in D$$

I.2. Fourier NO.

How to play in Fourier space?

$$(\mathcal{K}(a;\phi)v)(x) = \int_D \kappa_\phi(x-y)v(y)dy$$

Fourier transform

$$\hat{\mathcal{K}}v(k) = \hat{\kappa}(k)\hat{v}(k)$$

Continuous → Discrete

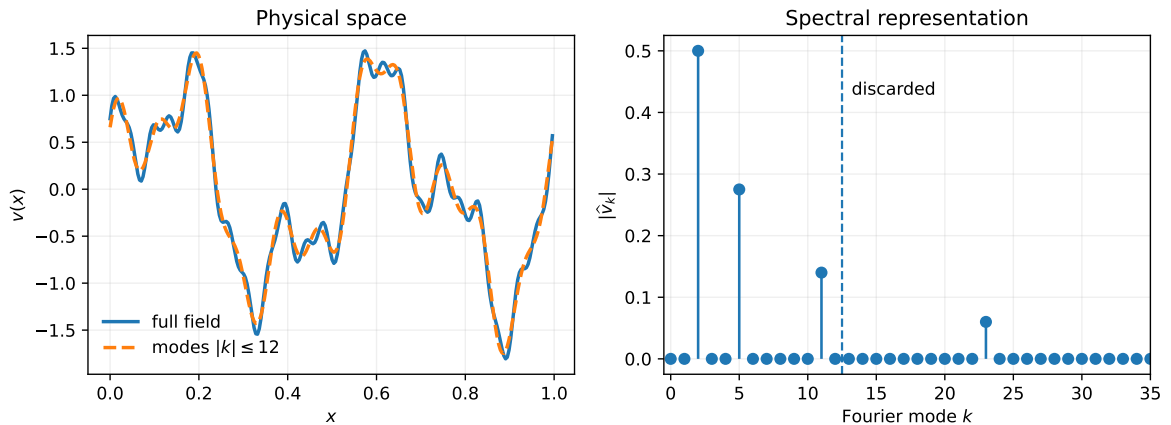
$$\hat{\mathcal{K}}v(k) = R_k\hat{v}(k)$$

$\hat{v}(k) \in \mathbb{C}^{d_v}$ being the Fourier transform of $v(y) \in \mathbb{R}^{d_v}$

$R_k \in \mathbb{C}^{d_v \times d_v}$

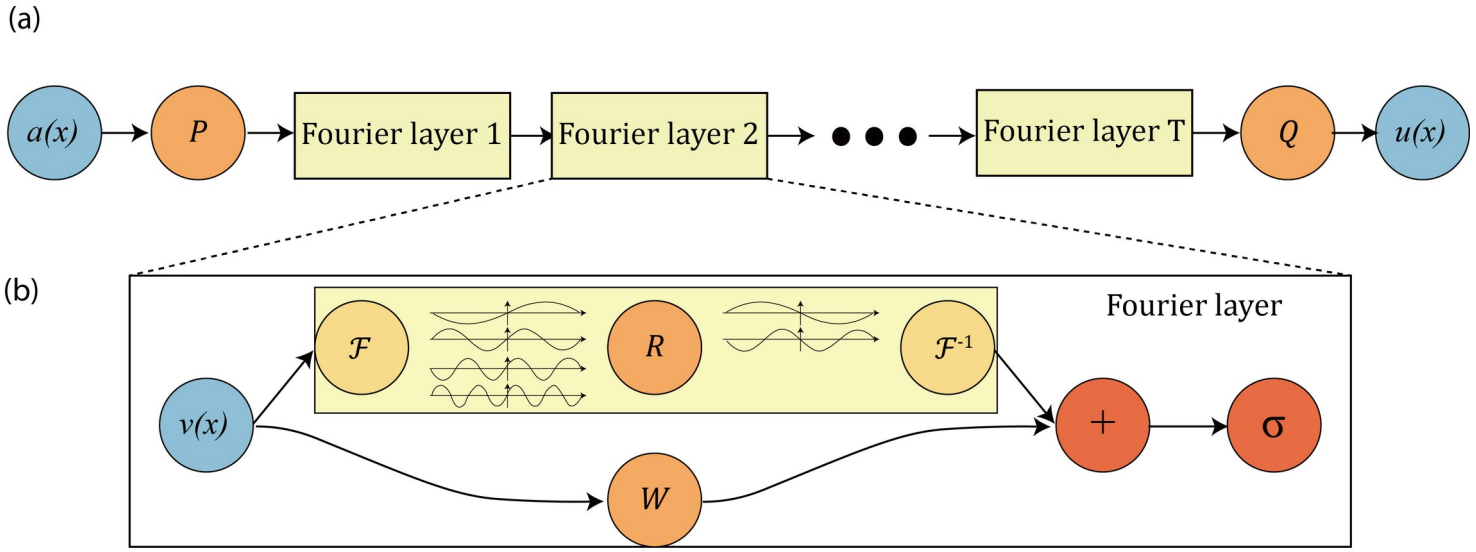
Let's parametrize the kernel in Fourier space $\mathcal{F}(\kappa_\phi) := R_\phi$

Implementation. *FFT is for regular grid, how to handle unregular ones?*



Fourier transform
→ keep selected modes
→ Linear matrix mixing per mode
→ inverse Fourier transform

I.2. Fourier Neural Operator.



(a) The full architecture of neural operator: start from input a. 1. Lift to a higher dimension channel space by a neural network P. 2. Apply four layers of integral operators and activation functions. 3. Project back to the target dimension by a neural network Q. Output u. **(b) Fourier layers**: Start from input v. On top: apply the Fourier transform $\mathcal{F}$; a linear transform R on the lower Fourier modes and filters out the higher modes; then apply the inverse Fourier transform $\mathcal{F}^{-1}$. On the bottom: apply a local linear transform W.

Figure 2: **top:** The architecture of the neural operators; **bottom:** Fourier layer.

Z. Li et al. (2021), *Fourier Neural Operator for Parametric Partial Differential Equations, ICLR.*

I. Neural Operator. Summary.

	PCA-Net	FNO
Assumption	Input/output solution families are approx. low-dimensional	The operator can be represented with a limited number of Fourier modes
Inductive bias	Low-rank linear representation	Spectral bias for non-local interactions
Hidden representation	Global coefficient vectors z_a, z_u	Spatial fields $\{V_j\}_{j=0}^{T}$

Do not blend:

Approximation.
Does a good parameter choice exist?

$\neq$

Learning.
Can finite data + optimization find one?

$\neq$

Extrapolation.
What happens outside the training distribution?

I. Neural Operator. Take-home.

1. A parametric PDE with BC and forcing terms naturally defines a **solution operator** i.e. function-to-function map.

2. Operator learning is most compelling in the **many-query** regime (expensive data generation can be amortized).

3. PCA-Net and FNO answer the same question by different ways: **compress to global coefficients** or **keep a field-valued latent state.**

Reference papers:

- N. Kovachki et al. (2023), *Neural Operator: Learning Maps Between Function Spaces with Applications to PDEs*, *JMLR*.

- Z. Li et al. (2021), *Fourier Neural Operator for Parametric Partial Differential Equations*, *ICLR*.

- V. Duruisseaux, J. Kossaifi, A. Anandkumar (2025), *Fourier Neural Operators Explained: A Practical Perspective*.

- K. Bhattacharya et al. (2021), *Model Reduction and Neural Networks for Parametric PDEs*.

- N. Boullé, A. Townsend (2023/24), *A Mathematical Guide to Operator Learning*.

Practicals 1 & 2:

- Discretization-independent
- Cross-resolution accuracy
- Extrapolation capabilities

Machine Learning for Physics. Part II

Henri Leroy, PhD Student (ML for granular) - *henri.leroy@inria.fr*

Univ. Grenoble Alpes, INRIA, MIAI | Artificial Intelligence & Mechanics (AIM)

Program:

I. Learning operators (1-2-3)

II. Geometric Deep Learning: Symmetry (4)

Questions?